



Лабораторная работа №8
по дисциплине
‘Программирование на Python’

Подготовила: Прозорова Полина Александровна
Группа: Р3120
Преподаватель: Жуков Николай Николаевич

Приложение для отслеживания курсов валют с подписками пользователей

Цель работы:

Освоить создание клиент-серверных приложений на Python с использованием стандартного HTTPServer, реализовать архитектуру MVC, интегрировать шаблонизатор Jinja2 и разработать систему подписок пользователей на валюты с отображением их динамики.

Описание предметной области:

Приложение позволяет пользователям подписываться на интересующие валюты и отслеживать их курсы в реальном времени.

Модели данных:

- Author - информация об авторе приложения (имя, группа)
- User - пользователь системы (ID, имя)
- Currency - валюта с метаданными (ID, цифровой код, символьный код, название, текущий курс, номинал)
- UserCurrency - связь пользователя с валютой (подписка)
- App - метайнформация о приложении (название, версия, автор)

Структура проекта:

```
laba8/
├── models/                # Модели данных
│   ├── __init__.py        # Экспорт моделей
│   ├── author.py          # Модель Author
│   ├── user.py            # Модель User
│   ├── currency.py        # Модель Currency
│   ├── user_currency.py   # Модель UserCurrency
│   └── app.py             # Модель App
├── templates/            # HTML шаблоны
│   ├── index.html         # Главная страница
│   ├── author.html        # Страница об авторе
│   ├── users.html         # Список пользователей
│   └── currencies.html    # Список валют
├── utils/                # Вспомогательные модули
│   ├── __init__.py
│   └── currencies_api.py  # Функция get_currencies
├── tests/                # Тесты
│   ├── test_templates.py  # Тесты шаблонов
│   ├── test_models.py     # Тесты моделей
│   ├── test_get_currencies.py # Тесты API
│   └── test_controller.py # Тесты контроллера
└── myapp.py              # Главный файл приложения
```

Назначение ключевых файлов:

- myapp.py - точка входа, содержит HTTP сервер и контроллер
- models/ - все сущности предметной области с валидацией
- templates/ - HTML шаблоны для отображения
- utils/currencies_api.py - модуль для получения курсов валют
- tests/ - модульные тесты для проверки корректности

Описание реализации:

1. Реализация моделей и свойств (геттеры/сеттеры)

Все модели реализованы с использованием инкапсуляции через приватные атрибуты и свойства (property):

```
class Currency:
    def __init__(self, id: int, num_code: str, char_code: str, name: str, value: float, nominal: int):
        self.__id = id
        self.__num_code = num_code
        # ... остальные атрибуты

    @property
```

```

def value(self) -> float:
    return self.__value

@value.setter
def value(self, value: float) -> None:
    if type(value) in (int, float) and value > 0:
        self.__value = value
    else:
        raise ValueError('Ошибка при задании курса валюты')

```

Каждый сеттер выполняет проверку типов и корректности значений, выбрасывая ValueError при нарушении.

2. Реализация маршрутов и обработка запросов

Используется стандартный BaseHTTPRequestHandler с простой маршрутизацией:

```

def do_GET(self):
    parsed = urlparse(self.path)
    path = parsed.path
    params = parse_qs(parsed.query)

    if path == '/':
        # Главная страница
    elif path == '/users':
        # Список пользователей
    elif path == '/user':
        # Информация о конкретном пользователе
        user_id = int(params.get('id', [1])[0])
    elif path == '/currencies':
        # Список валют
    elif path == '/currencies/update':
        # Обновление курсов

```

Маршруты:

- / - главная страница
- /author - информация об авторе
- /users - список всех пользователей
- /user?id=<id> - информация о конкретном пользователе
- /currencies - список валют
- /currencies/update - обновление курсов (редирект на /currencies)

3. Использование шаблонизатора Jinja2

Инициализация Environment выполняется один раз при старте приложения:

```

from jinja2 import Environment, PackageLoader, select_autoescape

env = Environment(
    loader=PackageLoader("myapp"), # Загрузчик ищет шаблоны в папке templates/
    autoescape=select_autoescape() # Автоматическое экранирование HTML
)

# Загрузка шаблонов
index_template = env.get_template("index.html")
users_template = env.get_template("users.html")
# ...

```

Рендеринг в обработчике:

```
html = users_template.render(
    myapp="CurrenciesListApp",
    author_name=author.name,
    group=author.group,
    users=users,
    current_user=current_user,
    user_currencies=user_currencies
)
```

4. Интеграция функции `get_currencies`

Функция `get_currencies` реализована как заглушка с тестовыми данными:

```
def get_currencies(currency_list=None):
    """Получает курсы валют (заглушка для лабораторной работы)."""
    currencies_data = {
        'USD': '92,50',
        'EUR': '99,80',
        'GBP': '115,20',
        # ... другие валюты
    }

    if currency_list:
        return {code: currencies_data[code] for code in currency_list if code in currencies_data}

    return currencies_data
```

Интеграция в контроллере:

```
elif path == '/currencies/update':
    data = get_currencies(['USD', 'EUR', 'GBP'])

    for currency in currencies:
        if currency.char_code in data:
            new_value = float(data[currency.char_code].replace(',', '.'))
            currency.value = new_value
```

Курсы обновляются по кнопке на странице `/currencies`.

5. Примеры работы приложения

1. Главная страница (`/`):

CurrenciesListApp

Автор проекта: Прозорова Полина

Группа: Р3120

- Главная
- Автор
- Пользователи
- Валюты

Добро пожаловать!

Это приложение для отслеживания курсов валют

2. Страница пользователей (/users):

CurrenciesListApp

Автор проекта: Прозорова Полина

Группа: Р3120

- Главная
- Автор
- Пользователи
- Валюты

Все пользователи

ID	Имя	Действия
1	Иван	Подробнее
2	Мария	Подробнее
3	Алексей	Подробнее

3. Страница конкретного пользователя (/user?id=):

CurrenciesListApp

Автор проекта: Прозорова Полина

Группа: Р3120

- Главная
- Автор
- Пользователи
- Валюты

Пользователь: Иван

ID: 1

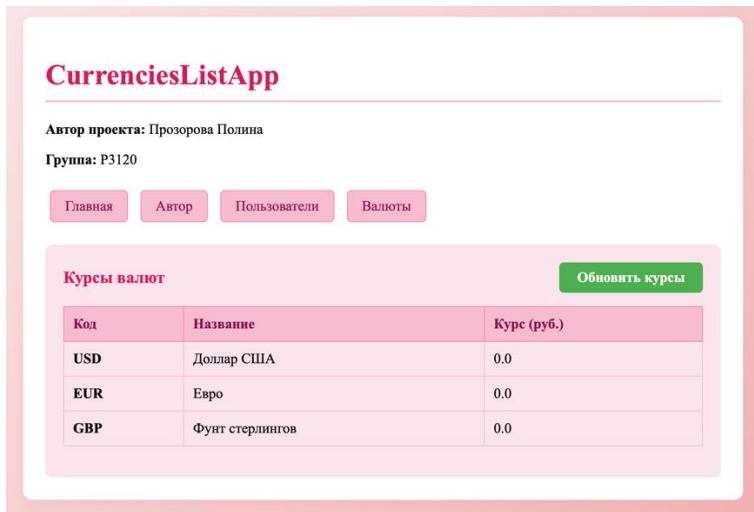
Подписки на валюты:

USD (Доллар США): 0.0 руб.
EUR (Евро): 0.0 руб.

Все пользователи

ID	Имя	Действия
1	Иван	Подробнее
2	Мария	Подробнее
3	Алексей	Подробнее

4. Страница валют (/currencies) до обновления:



6. Тестирование

Пример тестирования моделей:

```
#test_modules.py
import pytest
from laba8.models import Author, User, Currency, UserCurrency, App
```

```
class TestModels:
    """тестирование моделей - проверка геттеров, сеттеров и исключений"""

    def test_author_getters_setters(self):
        """проверка геттеров и сеттеров author"""
        author = Author("Полина", "Р3120")
        # геттеры
        assert author.name == "Полина"
        assert author.group == "Р3120"
        # сеттеры
        author.name = "Новое имя"
        author.group = "Р3121"
        assert author.name == "Новое имя"
        assert author.group == "Р3121"

    def test_author_exceptions(self):
        """проверка выброса исключений author при некорректных значениях"""
        author = Author("Тест", "Группа")
        with pytest.raises(ValueError):
            author.name = "" # пустое имя
        with pytest.raises(ValueError):
            author.group = "А" # слишком короткая группа

    def test_user_getters_setters(self):
        """проверка геттеров и сеттеров user"""
        user = User(1, "Иван")
        assert user.id == 1
        assert user.name == "Иван"
```

```

user.id = 2
user.name = "Петр"
assert user.id == 2
assert user.name == "Петр"

def test_user_exceptions(self):
    """проверка исключений user"""
    user = User(1, "Тест")
    with pytest.raises(ValueError):
        user.id = -1 # отрицательный ID
    with pytest.raises(ValueError):
        user.id = "строка" # не число
    with pytest.raises(ValueError):
        user.name = "Я" # слишком короткое имя

def test_currency_getters_setters(self):
    """проверка геттеров и сеттеров currency"""
    currency = Currency(1, "840", "USD", "Доллар", 92.5, 1)
    assert currency.id == 1
    assert currency.char_code == "USD"
    assert currency.value == 92.5
    currency.value = 95.0
    assert currency.value == 95.0

def test_currency_exceptions(self):
    """проверка исключений currency"""
    currency = Currency(1, "840", "USD", "Доллар", 92.5, 1)
    with pytest.raises(ValueError):
        currency.value = -10.0 # отрицательный курс
    with pytest.raises(ValueError):
        currency.char_code = "US" # неправильная длина

```

Результат работы:

```

===== test session starts =====
collecting ... collected 6 items

laba8/tests/test_modules.py::TestModels::test_author_getters_setters PASSED [ 16%]
laba8/tests/test_modules.py::TestModels::test_author_exceptions PASSED [ 33%]
laba8/tests/test_modules.py::TestModels::test_user_getters_setters PASSED [ 50%]
laba8/tests/test_modules.py::TestModels::test_user_exceptions PASSED [ 66%]
laba8/tests/test_modules.py::TestModels::test_currency_getters_setters PASSED [ 83%]
laba8/tests/test_modules.py::TestModels::test_currency_exceptions PASSED [100%]

===== 6 passed in 0.02s =====

Process finished with exit code 0

```

Пример тестирования контроллера:

```

#test_controller.py
import pytest
from urllib.parse import parse_qs, urlparse

```

```

class TestController:
    """тестирование контроллера"""

```

```

def test_root_route_logic(self):
    """проверка логики обработки маршрута / (главная)"""
    test_path = '/'
    if test_path == '/':
        assert True #маршрут обработан
    else:
        assert False

def test_users_route_logic(self):
    """Проверка маршрута /users"""
    test_path = '/users'

    if test_path == '/users':
        # должен вернуть список пользователей
        assert test_path.endswith('users')
    else:
        assert False

def test_currencies_route_logic(self):
    """Проверка маршрута /currencies"""
    test_path = '/currencies'

    if test_path == '/currencies':
        # должен вернуть список валют
        assert 'currencies' in test_path
    else:
        assert False

def test_user_query_parameter_parsing(self):
    """проверка обработки query-параметров /user?id=..."""
    test_url = '/user?id=2'
    parsed = urlparse(test_url)
    params = parse_qs(parsed.query)
    user_id = int(params.get('id', [1])[0])

    assert user_id == 2

    # тест с несколькими параметрами
    test_url2 = '/user?id=3&name=test'
    parsed2 = urlparse(test_url2)
    params2 = parse_qs(parsed2.query)
    user_id2 = int(params2.get('id', [1])[0])
    assert user_id2 == 3

def test_default_id_when_no_parameter(self):
    """проверка значения по умолчанию при отсутствии параметра"""
    test_url = '/user' # без параметра
    parsed = urlparse(test_url)
    params = parse_qs(parsed.query)

    # В вашем коде: int(params.get('id', [1])[0])
    default_id = int(params.get('id', [1])[0])

    assert default_id == 1 # значение по умолчанию

def test_invalid_id_parameter_handling(self):

```



```

"""проверка обработки неверного параметра ID"""
test_url = '/user?id=abc'
parsed = urlparse(test_url)
params = parse_qs(parsed.query)
assert 'id' in params
assert params['id'][0] == 'abc'

if __name__ == "__main__":
    pytest.main([__file__, "-v"])

```

Результат работы тестов:

```

===== test session starts =====
collecting ... collected 6 items

laba8/tests/test_controller.py::TestController::test_root_route_logic PASSED [ 16%]
laba8/tests/test_controller.py::TestController::test_users_route_logic PASSED [ 33%]
laba8/tests/test_controller.py::TestController::test_currencies_route_logic PASSED [ 50%]
laba8/tests/test_controller.py::TestController::test_user_query_parameter_parsing PASSED [ 66%]
laba8/tests/test_controller.py::TestController::test_default_id_when_no_parameter PASSED [ 83%]
laba8/tests/test_controller.py::TestController::test_invalid_id_parameter_handling PASSED [100%]

===== 6 passed in 0.01s =====

Process finished with exit code 0

```

Пример тестирования get_currencies:

```

#test_get_currencies.py
import pytest
from laba8.utils.currencies_api import get_currencies

```

```

class TestGetCurrencies:
    """тестирование функции get_currencies"""

    def test_correct_currencies_retrieval(self):
        """проверка корректного получения курсов"""
        result = get_currencies(['USD', 'EUR'])
        assert result == {'USD': '92,50', 'EUR': '99,80'}

    def test_get_mixed_currencies(self):
        """смесь существующих и несуществующих"""
        result = get_currencies(['USD', 'XYZ', 'EUR'])
        assert result == {'USD': '92,50', 'EUR': '99,80'}
        assert 'XYZ' not in result

    def test_nonexistent_currency_returns_empty(self):
        """проверка отсутствия валюты - возвращает пустой словарь"""
        result = get_currencies(['XYZ', 'ABC']) # несуществующие
        assert result == {} # пустой словарь

    def test_currency_format(self):
        """проверка формата данных"""
        result = get_currencies(['USD'])
        value = result['USD']
        assert isinstance(value, str)
        assert ',' in value

```

```
# преобразуем в число
assert float(value.replace(',', '.')) == 92.5

if __name__ == "__main__":
    pytest.main([__file__, "-v"])

Результат работы тестов:

===== test session starts =====
collecting ... collected 4 items

laba8/tests/test_get_currencies.py::TestGetCurrencies::test_correct_currencies_retrieval PASSED [ 25%]
laba8/tests/test_get_currencies.py::TestGetCurrencies::test_get_mixed_currencies PASSED [ 50%]
laba8/tests/test_get_currencies.py::TestGetCurrencies::test_nonexistent_currency_returns_empty PASSED [ 75%]
laba8/tests/test_get_currencies.py::TestGetCurrencies::test_currency_format PASSED [100%]

===== 4 passed in 0.01s =====

Process finished with exit code 0
```

7. Выводы:

Проблемы при реализации:

1. Импорт модулей - возникли сложности с правильной структурой пакетов и импортами между модулями
2. Маршрутизация - стандартный BaseHTTPRequestHandler требует ручной реализации маршрутизации
3. Тестирование - сложности с организацией тестов из-за циклических импортов
4. Формат данных - необходимость преобразования формата курсов (запятая → точка)

Применение принципов MVC:

- Model - классы в папке models/ с бизнес-логикой и валидацией
- View - шаблоны Jinja2 в папке templates/ только для отображения
- Controller - SimpleHTTPRequestHandler в myapp.py обрабатывает запросы и связывает Model с View

Новые знания:

1. HTTPServer - освоено создание веб-сервера без фреймворков, обработка запросов и маршрутизация
2. Jinja2 - изучена шаблонизация с наследованием, циклами и условиями, правильная инициализация Environment
3. Архитектура MVC - применено разделение ответственности между компонентами
4. Тестирование – написаны тесты для моделей и API с использованием pytest
5. Работа с API - создан модуль для получения данных с возможностью замены заглушки на реальное API

Лабораторная работа позволила освоить создание полноценного веб-приложения на чистом Python с архитектурой MVC. Получены практические навыки работы с HTTP сервером, шаблонизацией, валидацией данных и модульным тестированием. Приложение может быть расширено добавлением базы данных и реального API курсов валют.